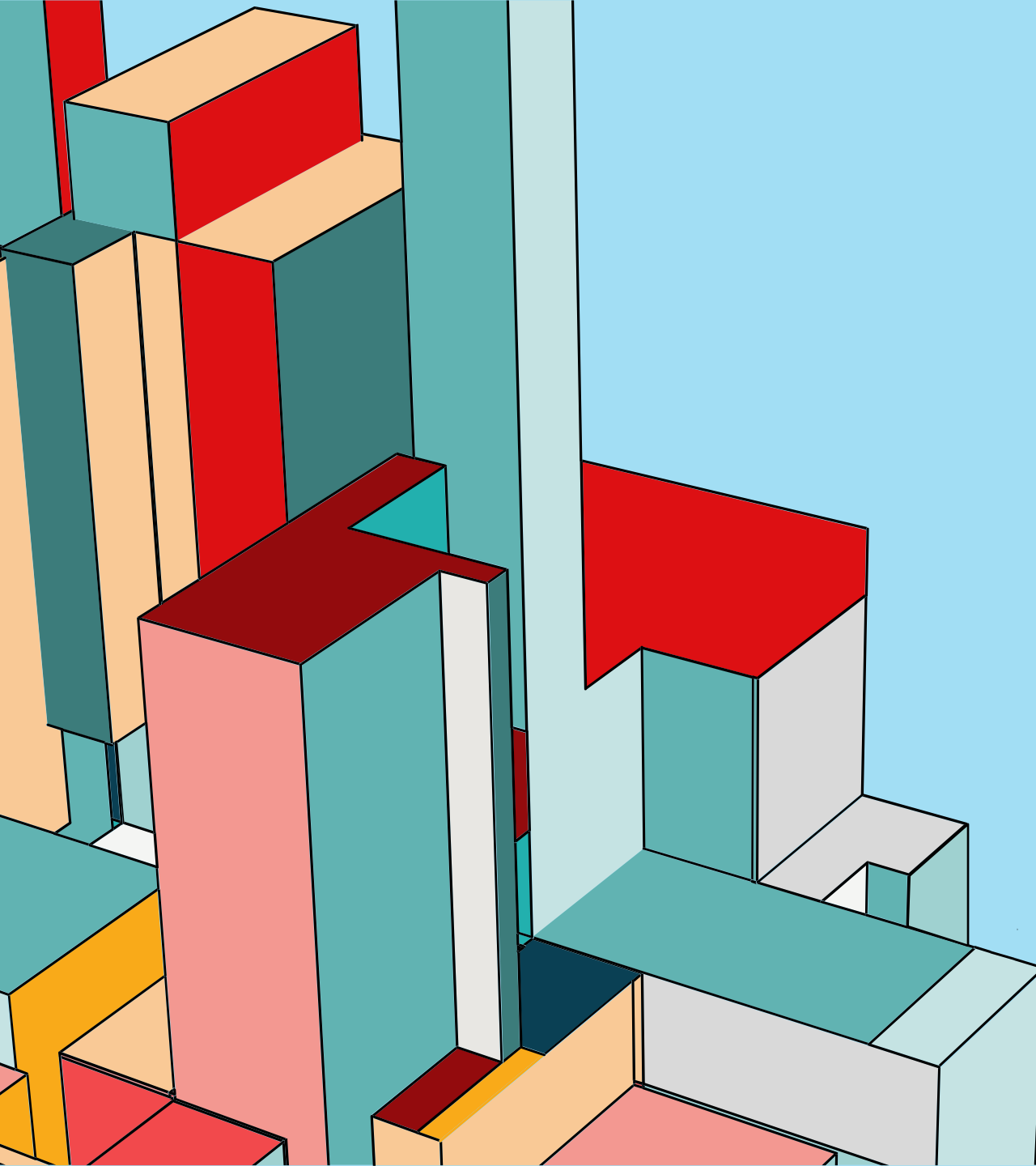


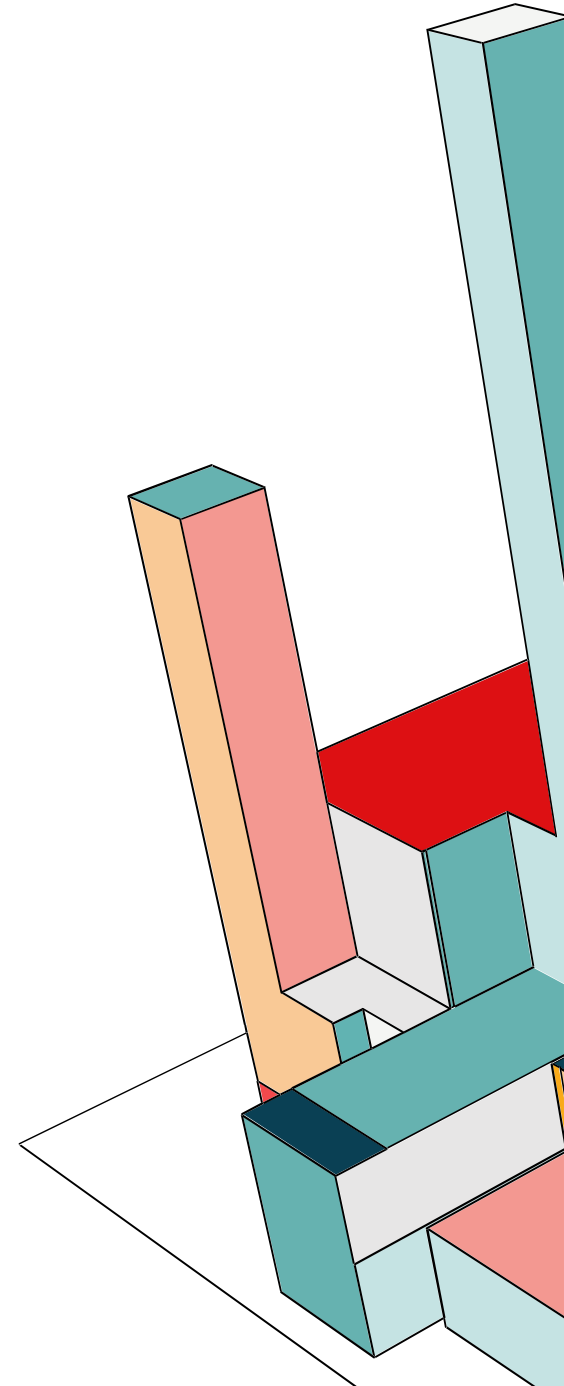


LECTURE 2: THE ANALYSIS FRAMEWORK

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

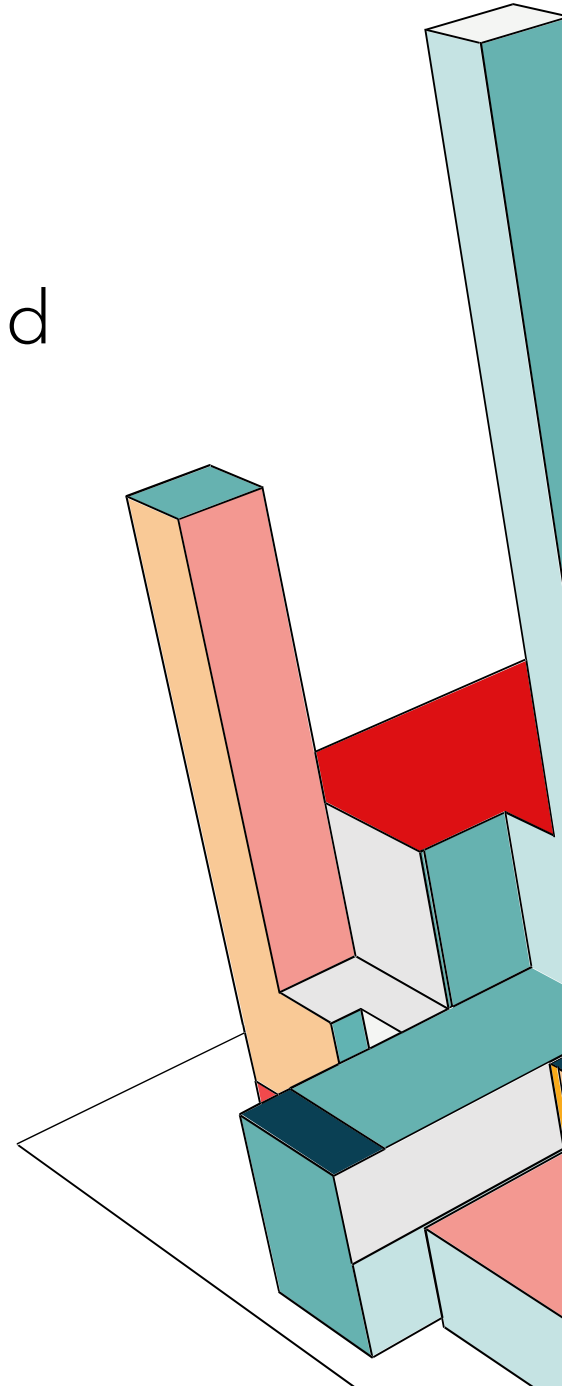
TODAY'S AGENDA

- General approach
- Asymptotic notation
- Non-recursive functions
- Recursive functions



HOW FAST DOES IT RUN?

- We can (and will) talk about both *time complexity* and *space complexity*, but the vast majority of time we're more interested in speed.
 - There are two main reasons for this: We can usually do more about runtime than memory usage, and (until the recent crunch due to greedy AI companies) memory has been relatively cheap.
- There are a lot of factors that go into this.



ACTUAL RUNTIME VS. OPERATIONS

As any gamer will tell you, speed of execution is intimately linked to the hardware we're running on. But we don't normally know that when we're writing an algorithm – it could be running on an iPhone 17 Pro or on a potato.

What we can generally safely assume is that individual operations will take the same amount of the time – e.g. every multiplication takes some constant time c .

Thus, we'll just count operations. Even if those operations take different amounts of times, as long as they're all constant, it won't affect our analysis.

ORDER OF GROWTH

You have a super-complicated Excel spreadsheet that takes an hour to calculate all values. You buy a new computer that is 10 times as fast. How long do you expect the computation to take now?

1/10 as long, so about six minutes.

Same situation, but your dataset now has twice as many rows as it did before. Now how long will it take?

Uh... we have no idea. Depends on what it's doing.

SUBMITTY POLL #1

Remember bubble sort? You run a bubble sort on 10,000 items and it takes 1 minute. How long do you expect it to take on 100,000 items?



A NOTE ABOUT SIZE OF THE INPUT

Algorithms run on data. Data is ultimately stored as bits. Thus, our measure of the size of the input is bits. This has some implications, depending on the data structure.

- Any sort of array or list is simply the number of elements in the array times the size of each element.
- Similarly, we can just count nodes in a tree.
- A graph is trickier, because we might count vertices or edges. But for most interesting graphs, we should use edges.
- Single numeric values are the biggest trap. If we input a value like 1,000,000, the length of that input is only 20 bits.

SIZE ISN'T EVERYTHING

Consider this pseudocode:

```
def find(A,n,k):  
    for i = 0 to n-1:  
        if A[i] == k:  
            return i  
    return -1
```

How long will this take to run?

... _(ツ)_/

WORST-, BEST-, AND AVERAGE-CASE

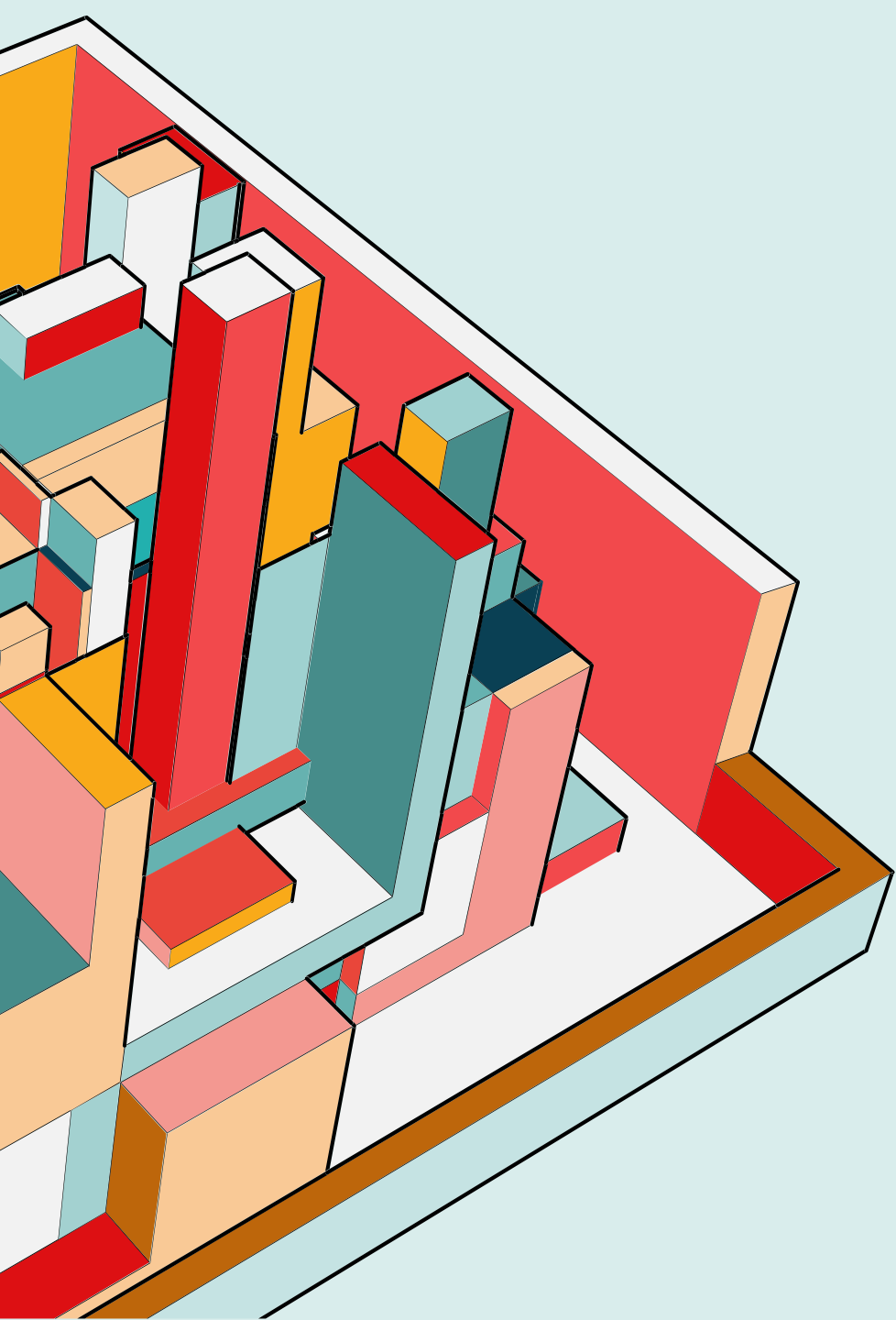
The find algorithm on the last slide might look at one array element or all of them! Which should we consider?

Average-case behavior gives us a good sense of what we can expect to happen, but it is often harder to calculate.

So usually we look at *worst-case* behavior. That way, we can guarantee performance will be at least as good as we say.

On the other hand, if we consider best-case behavior and it's still terrible, we can skip the rest of the analysis.





ASYMPTOTIC ANALYSIS

ASYMPTOTIC NOTATION

Our challenge is to take the notion of “order of growth” and come up with a systematic, mathematical definition.

To fulfill this role, we use *asymptotic notation*. One of these you are already familiar with: big- O notation. There are four others: little- o , big- Θ , big- Ω , and little- ω .

All of them are comparisons of the order of growth of two functions as the input size grows very large.

USING ASYMPTOTIC NOTATION

Asymptotic notation is generally written in one of two ways, depending on what book you use:

- $f(n) = O(g(n))$ or
- $f(n) \in O(g(n))$

Both of these mean the same thing!

The latter is better notation, since $O(n^2)$ represents the set of functions whose order of growth is no worse than quadratic. But we'll use the former, both because it is more common and because it is easier to typeset.

USING ASYMPTOTIC NOTATION

To a first approximation, the five asymptotic notations are direct analogs of the five normal comparisons, as shown at right.

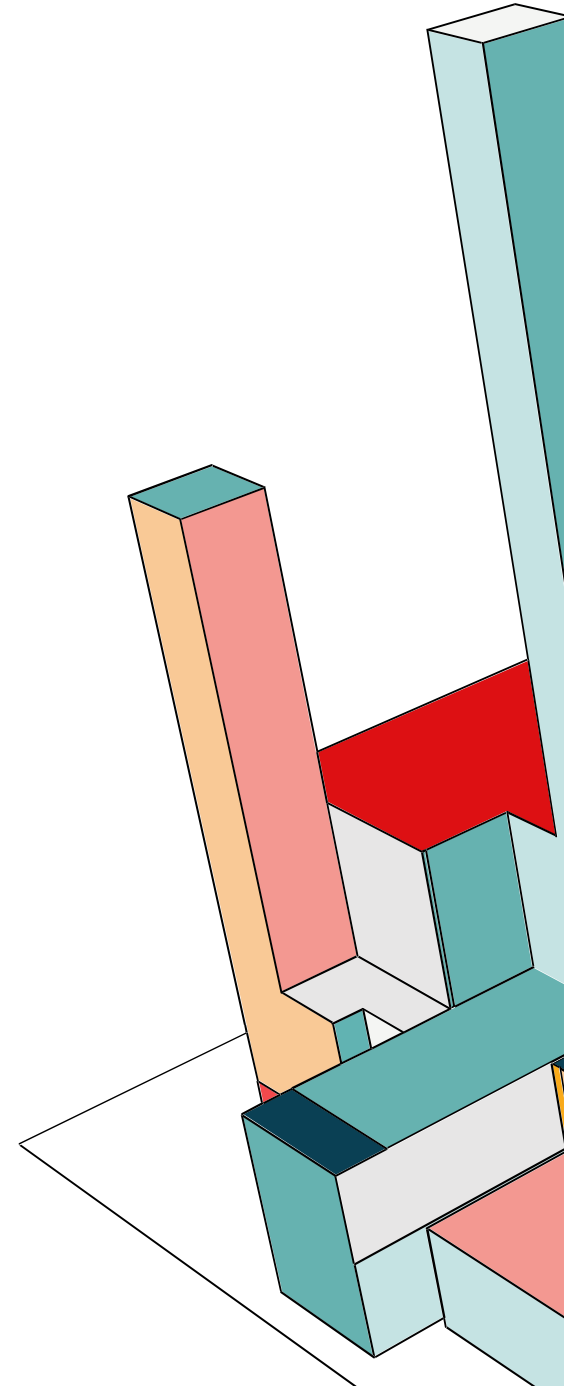
For example, if we say $f(n) = o(g(n))$, we mean that the order of growth of f is strictly slower than the order of growth of g .

Meanwhile, $f(n) = \Theta(g(n))$ means that f and g have the same order of growth.

o	$<$
O	$\leq$
Θ	$=$
Ω	$\geq$
ω	$>$

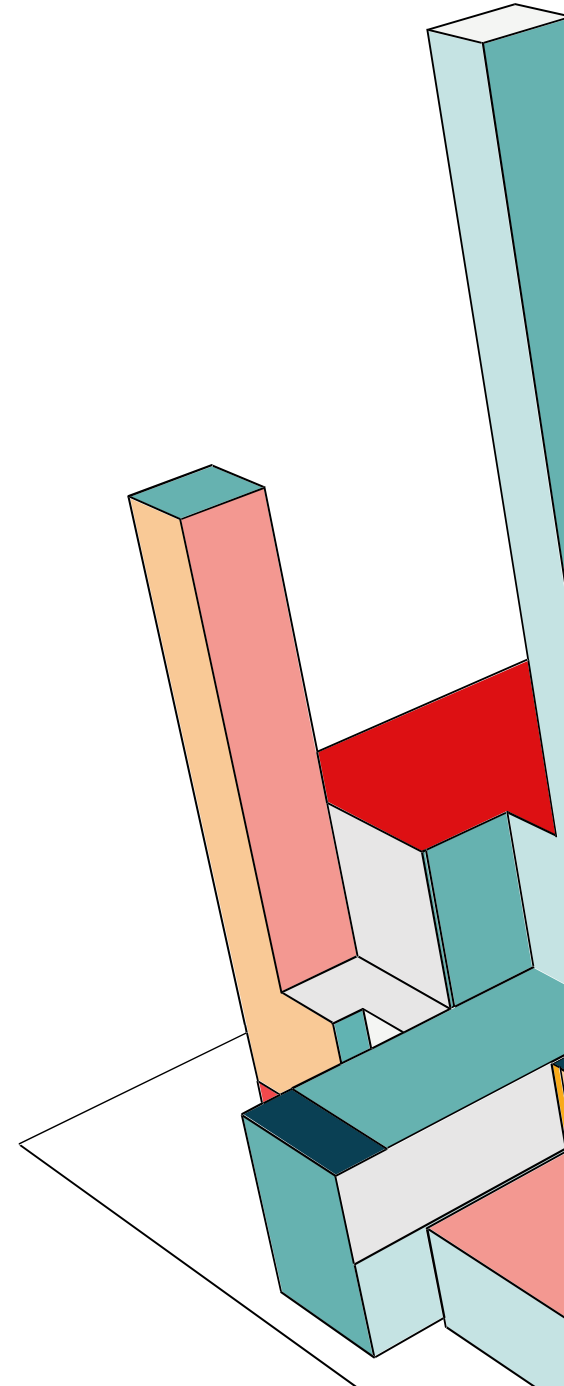
TRY IT NOW

- Rate each of these statements as true or false:
 - $n^2 + 5n + 100 = O(n^3)$
 - $n^2 + 5n + 100 = o(n^3)$
 - $n^2 + 5n + 100 = O(n \lg n)$
 - $7n^3 + 30n^2 + 100000n = \Omega(2^n)$
 - $2.5^n = \Theta(3^n)$
 - $100 \log_2 n = \Theta(\log_3 n)$
 - $\sqrt{n} = O(\lg n)$



TRY IT NOW

- Rate each of these statements as true or false:
 - $n^2 + 5n + 100 = O(n^3)$ -- TRUE
 - $n^2 + 5n + 100 = o(n^3)$ -- TRUE
 - $n^2 + 5n + 100 = O(n \lg n)$ -- FALSE
 - $7n^3 + 30n^2 + 100000n = \Omega(2^n)$ -- FALSE
 - $2.5^n = \Theta(3^n)$ -- FALSE
 - $100 \log_2 n = \Theta(\log_3 n)$ -- TRUE
 - $\sqrt{n} = O(\lg n)$ -- FALSE (but how do we know??)



The asymptotic menagerie

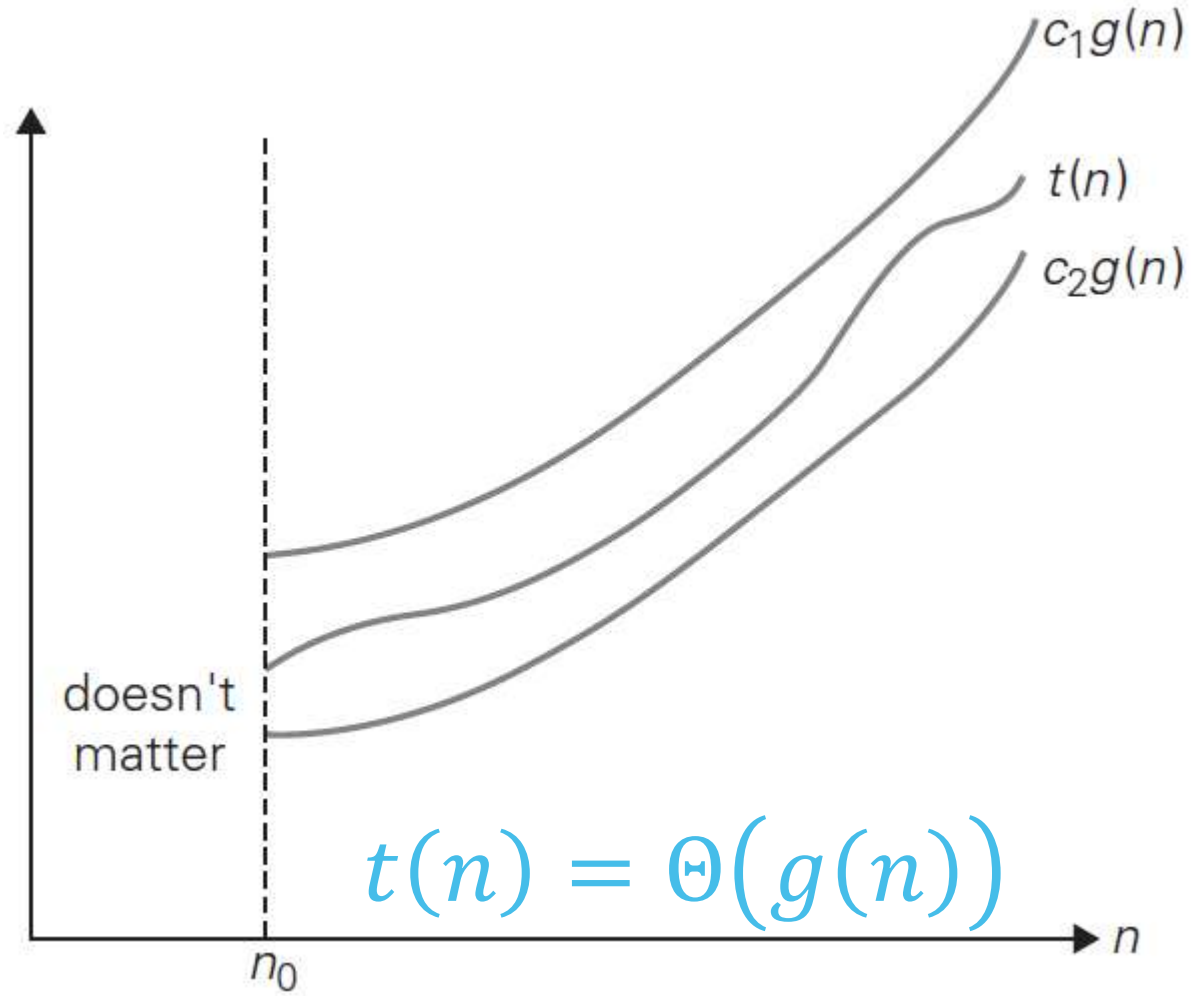
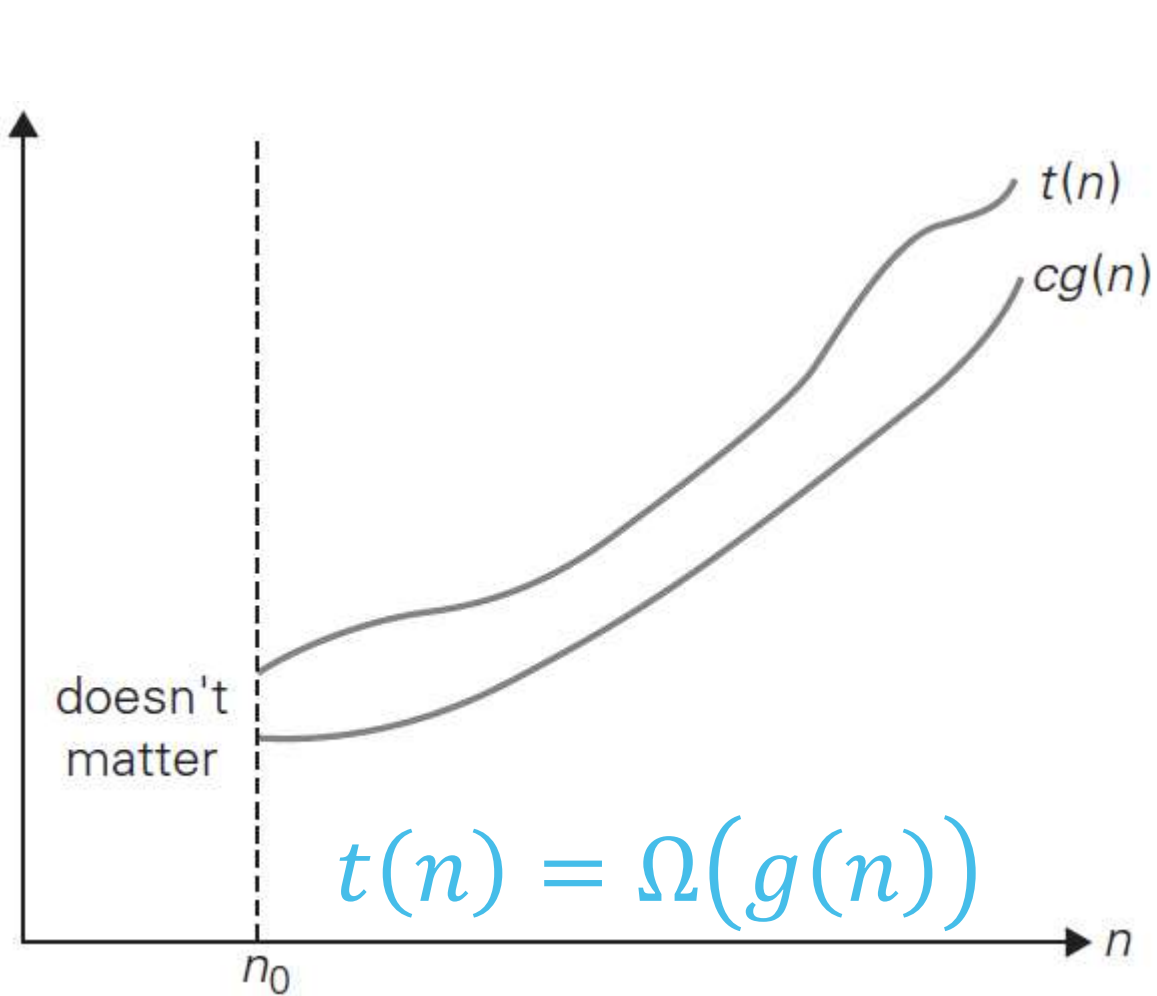
- The common classes of functions we work with are:
 - $O(1)$ - constant (*array access*)
 - $O(\log n)$ - logarithmic (*binary search in a sorted array*)
 - $O(n)$ - linear (*search in an unsorted array*)
 - $O(n \log n)$ - loglinear (*good sorting algorithms like mergesort*)
 - $O(n^2)$ - quadratic (*poor sorting algorithms like bubble sort*)
 - $O(n^c)$ - higher-order polynomials (*some parsing algorithms*)
 - $O(n^{\log n})$ - quasipolynomial (*old primality test*)
 - $O(2^n)$ - exponential (*Traveling Salesman Problem*)
 - $O(n!)$, $O(n^n)$, $O(n^{2^n})$, and even worse - Don't go there.

FORMAL DEFINITIONS

Starting with the bigs:

- $f(n) = O(g(n))$ means that there exists some constant $c_1 \geq 0$ and some input size n_0 such that $\forall n > n_0, f(n) \leq c_1 \cdot g(n)$
- $f(n) = \Omega(g(n))$ means that there exists some constant $c_2 \geq 0$ and some input size n_0 such that $\forall n > n_0, f(n) \geq c_2 \cdot g(n)$
 - Notice that this means $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$
- $f(n) = \Theta(g(n))$ simply means both of the above at the same time. That is, there are constants $c_1, c_2 > 0$ and an input size n_0 such that $\forall n \geq n_0, c_2 \cdot g(n) \leq f(n) \leq c_1 \cdot g(n)$.

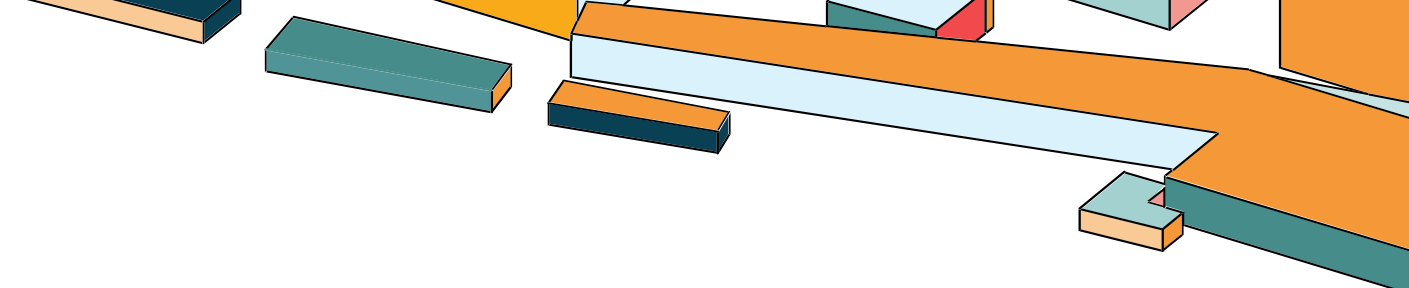
ASYMPTOTICS, GRAPHICALLY



FORMAL DEFINITIONS

The littles are actually stronger statements:

- $f(n) = o(g(n))$ means that for every constant $c \geq 0$, there is some input size n_0 such that $\forall n > n_0, f(n) < c \cdot g(n)$
- $f(n) = \omega(g(n))$ means that for every constant $c \geq 0$, there is some input size n_0 such that $\forall n > n_0, f(n) > c \cdot g(n)$
 - Thus, $f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n))$
- The idea here is that the larger function completely dominates the smaller one - they are in different asymptotic classes.
- Note also that $f(n) = o(g(n)) \Rightarrow f(n) = O(g(n))$ (ditto for ω/Ω)

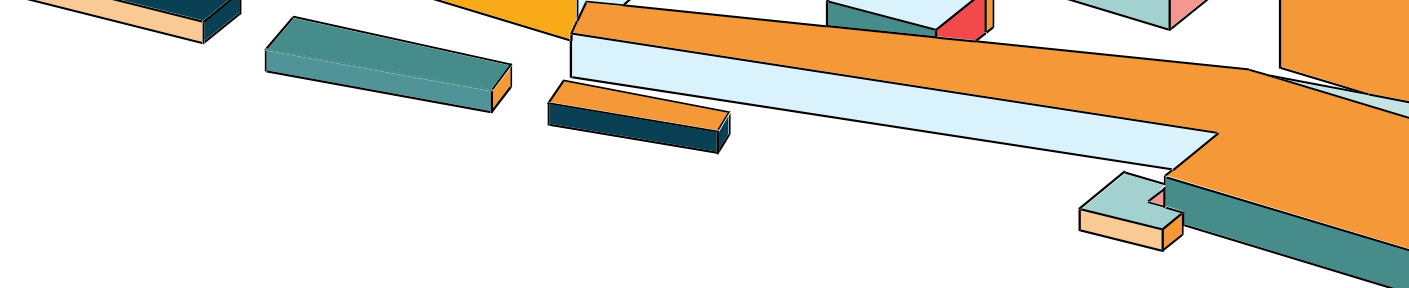
A collection of 3D rectangular blocks in various colors (orange, teal, light blue, dark blue) arranged in a scattered pattern in the top right corner of the slide.

TRY IT NOW

Prove the following statements. (This means you must give the relevant constants.)

- $4n^2 + 50n + 1000 = \Theta(n^2)$

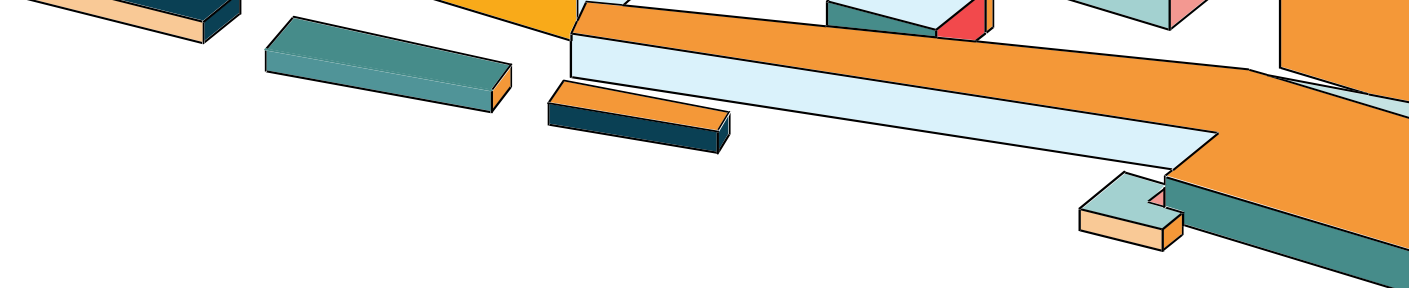
- $n^{10} = O(2^n)$



TRY IT NOW

Prove the following statements. (This means you must give the relevant constants.)

- $4n^2 + 50n + 1000 = \Theta(n^2)$
 - 4 obviously works for the lower bound constant. We can use 5 for the upper one.
 - How big does n have to be before $5n^2$ is larger than our function?
 - Clearly more than 50. But we don't need to get it precise.
 - $n_0 = 1000$ works just fine.
- $n^{10} = O(2^n)$
 - n_0 clearly should be more than 10.
 - Go big or go home! $n_0 = 1000$ is overkill, and lets us use $c = 1$.

A collection of 3D rectangular blocks in various colors (orange, teal, light blue, dark blue) arranged in a scattered, overlapping pattern in the top right corner of the slide.

LITTLE-O PROOFS

Note that proving this is a different animal.

- $n^{10} = o(2^n)$

In this case, we need to show that we can find an n_0 for any $c > 0$ that we might select. So our n_0 will actually be a function of that constant c .

Or, we could use a different approach...

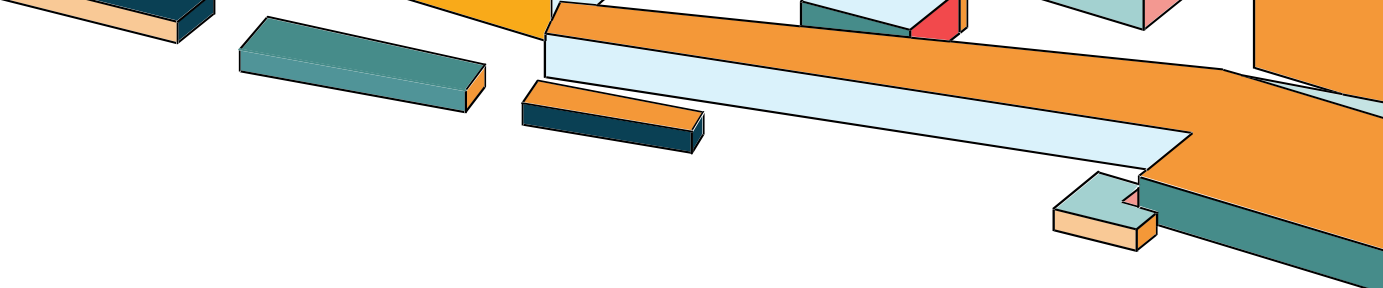
THE LIMIT RULE

When comparing two functions directly, one can take the limit of the quotient.

If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \dots$

- 0, then $f(n) = o(g(n))$ -- f is dominated by g
- c , then $f(n) = \Theta(g(n))$ -- the two functions are the same order
- ∞ , then $f(n) = \omega(g(n))$ -- f dominates g

The advantage of this approach is that we can take advantage of calculus techniques for dealing with limits...



LITTLE-O PROOFS

Note that proving this is a different animal.

- $n^{10} = o(2^n)$

Ok, what's $\lim_{n \rightarrow \infty} \frac{n^{10}}{2^n}$?

We “know” it's 0, but how do we prove it? We can get an assist from L'Hôpital's Rule, which says the limit of two functions as $n \rightarrow \infty$ equals the limit of the derivatives of those functions.

$$\lim_{n \rightarrow \infty} \frac{n^{10}}{2^n} = \lim_{n \rightarrow \infty} \frac{10n^9}{\ln 2 \cdot 2^n}$$

$$= \lim_{n \rightarrow \infty} \frac{90n^8}{(\ln 2)^2 \cdot 2^n} = \dots = \lim_{n \rightarrow \infty} \frac{10!}{(\ln 2)^{10} \cdot 2^n} = 0$$

NON-RECURSIVE FUNCTIONS



COUNTING OPERATIONS

Q: If our procedure is not recursive, then what will generate most of our operations?

A: The loops.

So if we want to count the operations, and we're really only worried about the order of growth, we would do well to focus on the *most expensive operation of the innermost loop*.

Caveat: We need to double-check that the number of times this operation is executed depends only upon the input size. If not, we'll need to break out best-, worst-, and average-case.

COUNTING OPERATIONS

Once we've established the critical operation, we just need summations for each enclosing loop, and then use rules for summations to try to produce a closed form solution.

Key rules:

$$\sum_{i=a}^b c x_i = c \sum_{i=a}^b x_i$$

$$\sum_{i=a}^b (x_i \pm y_i) = \left(\sum_{i=a}^b x_i \right) \pm \left(\sum_{i=a}^b y_i \right)$$

$$\sum_{i=a}^b 1 = b - a + 1$$

$$\sum_{i=0}^n i = \sum_{i=1}^n i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} = \Theta(n^2)$$

BASIC EXAMPLE

Here's some pseudocode to find the largest element of an array.

```
def MaxElement(A):  
    max = A[0]  
    for i = 1 to n-1:  
        if A[i] > max:  
            max = A[i]  
    return max
```

$$\sum_{i=1}^{n-1} 1 = n - 1 - 1 + 1 = \Theta(n)$$

There's one loop. The critical operation is thus either the comparison or the assignment of a new max. Since the comparison happens every iteration (and they are equally expensive), we should use the comparison.

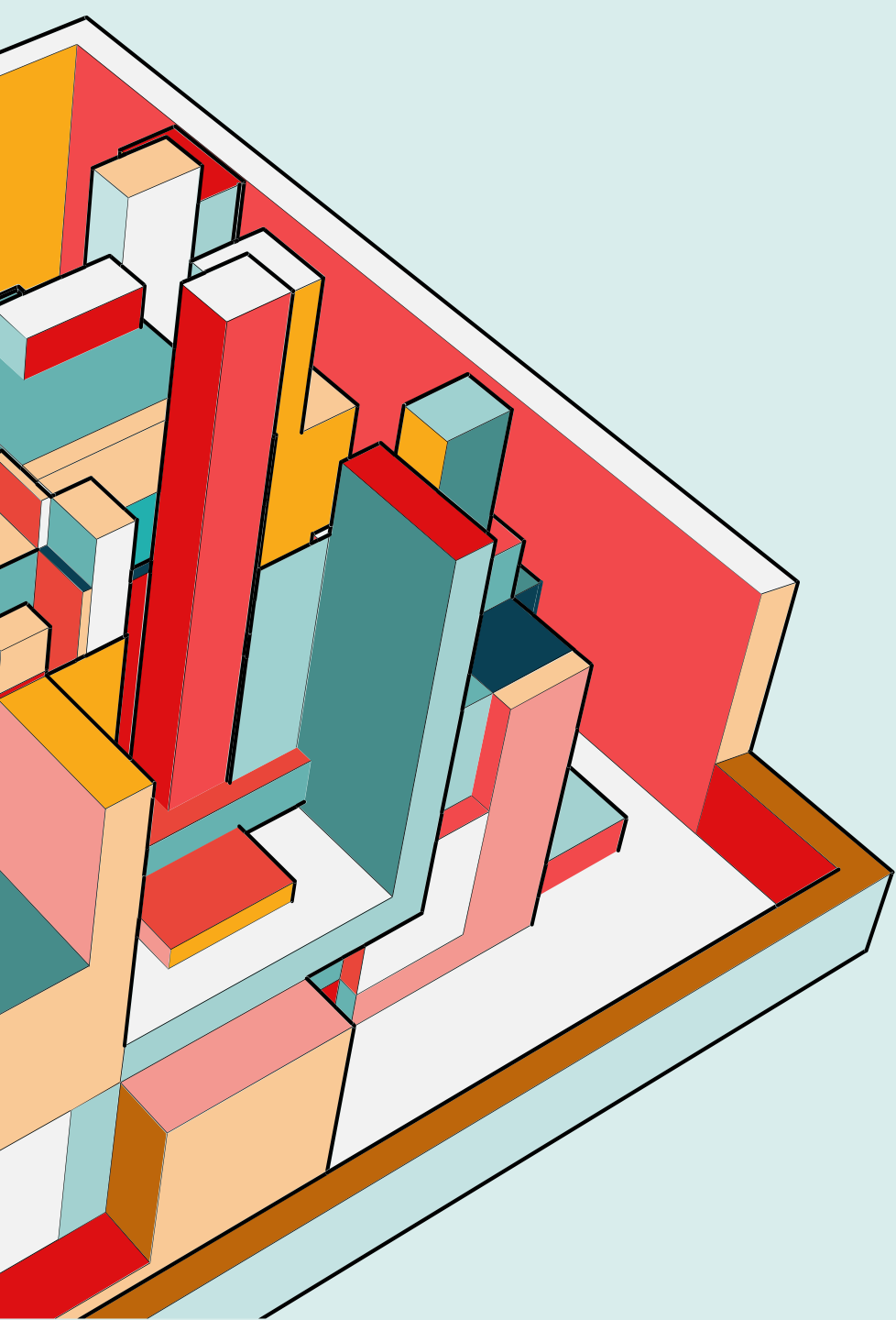
ANOTHER EXAMPLE

This time, let's check if all elements of an array are unique.

```
def AllUnique(A):  
    for i = 0 to n-2:  
        for j = i+1 to n-1:  
            if A[i] == A[j] return false  
    return true
```

Create the summation for this algorithm now.

Then find the closed form.



RECURSIVE FUNCTIONS

COUNTING FUNCTION CALLS

With recursive algorithms, there usually isn't a loop. Instead, the main thing we need to count are the recursive calls themselves, as each one usually takes a constant amount of time.

A very simple recursive algorithm is one for computing factorial.

```
def factorial(n):  
    if n > 1:  
        return n * factorial(n-1)  
    return 1
```

CREATING RECURRENCE RELATIONS

```
def factorial(n):  
    if n > 1:  
        return n * factorial(n-1)  
    return 1
```

To count the recursive calls, we want to create a recurrence relation: $F(n) = 1 + F(n - 1)$.

Base case: $F(1) = 1$

The 1 is because we perform one multiplication at each call. The $F(n - 1)$ is because we're calling **factorial(n-1)**.

Hopefully, it's obvious that this is linear, but let's look at why.

FORWARD AND BACKWARD SUBSTITUTION

$$F(n) = 1 + F(n - 1); F(1) = 1$$

Forward substitution – start at the bottom: $F(1) = 1$; $F(2) = 1 + F(1) = 2$; $F(3) = 1 + F(2) = 3$; ...

Backward substitution – start at the top: $F(n) = 1 + F(n - 1) = 1 + (1 + F(n - 2)) = 1 + (1 + (1 + F(n - 3))) = \dots$

In both cases, this is mostly a “find the pattern” approach, as opposed to a rigorous proof, although we can effectively string out either one until we get to the end.

RECURSIONS THAT DECREASE BY ONE

$$F(n) = g(n) + F(n - 1)$$

Backward substitution will show us that anything that looks like this will simply sum up $g(n) + g(n - 1) + \dots + g(1) = \sum_{i=1}^n g(i)$.

For most g , we can find a closed-form solution of this sum.

Other recursions are trickier...

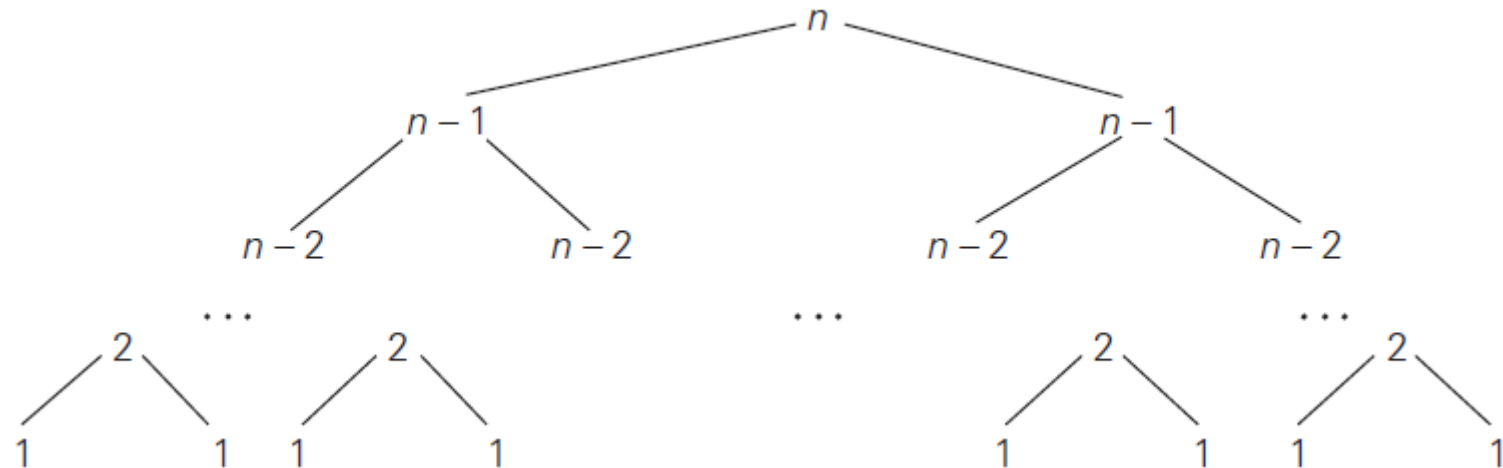
MULTIPLE RECURSIVE CALLS

```
def Hanoi(n,x,y,z):  
    Hanoi(n-1,x,z,y)  
    Move n from x to z  
    Hanoi(n-1,y,x,z)
```



$$F(n) = F(n-1) + 1 + F(n-1) = 1 + 2F(n-1); F(1) = 1$$

... now what?



TOWERS OF HANOI

$$F(n) = 1 + 2F(n - 1); F(1) = 1$$

Well, $F(1) = 1$. $F(2) = 1 + 2F(1) = 3$. $F(3) = 1 + 2F(2) = 7$...

Sure *looks* exponential.

In general, anything that makes multiple recursive calls is going to be exponential unless we can make the problem a LOT smaller at each stage – decreasing by one isn't going to cut it.

THOUGHT FOR THE DAY

- " $O(n^2)$ is the sweet spot of badly scaling algorithms: fast enough to make it into production, but slow enough to make things fall down once it gets there." --Bruce Dawson
- [This blog post](#) details a lovely little bug that shipped with Windows 11, in which one of its UI threads would sometimes hang...
 - ... because it used a quadratic-time algorithm to place desktop icons nicely onto a grid ...
 - ... even if those icons weren't being displayed right now!

BEFORE YOU LEAVE

REMINDERS: No class Monday. HW #1
due Tuesday night.

